

PsicoTest — Guía de onboarding para colaboradores

Bienvenido/a al repositorio de PsicoTest. Este documento explica paso a paso lo que debe hacer un nuevo colaborador que clona el proyecto para dejarlo funcionando en su entorno local (con Docker) y cómo solucionar problemas comunes.

Requisitos previos

- Git
- Docker y Docker Compose
- Composer (si vas a instalar dependencias fuera del contenedor)
- Node.js y npm/yarn (opcional si trabajas en assets front)

1. Clonar el repositorio

-

PsicoTest — Guía de arranque y comandos (ordenados)

Esta guía concentra los pasos exactos y los comandos en el orden correcto para que un nuevo colaborador deje el proyecto funcionando localmente (con Docker). Está en español y contiene atajos y soluciones a problemas comunes.

Requisitos previos

- Git
- Docker y Docker Compose
- (Opcional) Composer y Node.js si prefieres no usar el contenedor para instalar dependencias

Resumen rápido (comandos mínimos en orden)

1. Clonar el repo

```
git clone <repo-url>
cd PsicoTest
```

2. Copiar y revisar variables de entorno

```
cp .env.example .env
# Edita `.env` si necesitas (DB_HOST, DB_USER, APP_URL, etc.)
```

3. Levantar servicios Docker (construye imágenes si cambias Dockerfile)

```
docker-compose up -d --build
```

4. Instalar dependencias PHP (dentro del contenedor) y optimizar autoload

```
docker-compose exec app composer install --no-interaction --prefer-dist --optimize-autoloader
docker-compose exec app composer dump-autoload -o
```

5. Frontend: generar iconos y compilar assets

```
npm install
npm run generate-icons # crea public/bootstrap-icons-list.json
npm run dev
```

6. Migraciones / seeders (desarrollo)

```
# Si puedes resetear la BD (desarrollo)
docker-compose exec app php artisan migrate:fresh --seed

# Si no quieres resetear la BD (aplicar migraciones pendientes)
docker-compose exec app php artisan migrate --seed
```

Nota importante (assets & iconos)

- Este proyecto usa un generador local para crear la lista de iconos de Bootstrap Icons usada por los pickers del frontend.
- Antes de ejecutar `npm run dev`, si es la primera vez o si has actualizado dependencias, ejecuta:

```
npm run generate-icons
```

Esto crea `public/bootstrap-icons-list.json`. Si `npm run dev` falla con errores relacionados a iconos o CSS, ejecutar el comando anterior suele resolverlo.

Si `npm run dev` sigue fallando con Exit Code 1, intenta construir para producción y revisar la salida:

```
npm run generate-icons  
npm run build
```

y revisa la salida de la terminal para el error concreto; copia el texto y pégalo en la issue o en la conversación para ayuda.

6. Generar clave de la aplicación (si aún no está)

```
docker-compose exec app php artisan key:generate
```

7. Ejecutar migraciones y seeders (desarrollo)

Si estás en desarrollo y puedes resetear la BD (recomendado para sincronizar esquema):

```
docker-compose exec app php artisan migrate:fresh --seed
```

Si NO quieres resetear la BD (aplicar migraciones pendientes):

```
docker-compose exec app php artisan migrate --seed
```

8. Limpiar y recargar cachés

```
docker-compose exec app php artisan config:clear  
docker-compose exec app php artisan cache:clear  
docker-compose exec app php artisan view:clear  
docker-compose exec app php artisan route:clear
```

o (mejor)

```
docker-compose exec app php artisan optimize:clear
```

9. Ajustar permisos si ves errores de escritura (views / cache)

```
docker-compose exec app chown -R www-data:www-data storage bootstrap/cache || true
docker-compose exec app chmod -R ug+rw storage bootstrap/cache || true
```

10. Comprobar que ADMIN_EMAILS funciona y reasignar admin si hace falta

```
# Ver la config leída
docker-compose exec app php artisan tinker --execute="var_export(config('app.admin_emails'))"

# Re-seed si necesitas forzar asignaciones a admin
docker-compose exec app php artisan db:seed --class=DatabaseSeeder

# O con tinker asignar admin manualmente
docker-compose exec app php artisan tinker
>>> $u = \App\Models\User::where('email','test@admin.com')->first();
>>> $u->syncRoles(['admin']);
>>> $u->is_active = true; $u->save();
```

Comandos útiles frecuentes (atajos)

- Levantar todo (build si hace falta): `docker-compose up -d --build`
- Parar: `docker-compose down`
- Ejecutar artisan dentro del contenedor: `docker-compose exec app php artisan <command>`
- Abrir un shell en el contenedor: `docker-compose exec app sh` (o `bash` si está disponible)
- Ejecutar tests: `docker-compose exec app ./vendor/bin/pest --colors` (o `vendor/bin/phpunit` si aplica)

Diagnóstico y resolución de problemas comunes

- Redis
 - Si ves `Class "Redis" not found` : o bien instala la extensión `phpredis` en el PHP que ejecuta Artisan, o cambia a `predis` con `composer require predis/predis` y `REDIS_CLIENT=predis` en `.env` .
 - Si ves `php_network_getaddresses: getaddrinfo for redis failed` : estás ejecutando Artisan en el host y `REDIS_HOST=redis` sólo existe dentro de la red Docker. Ejecuta Artisan dentro del contenedor o mapea el puerto Redis en `docker-compose.yml` .
- Middleware `perm`
 - Si ves `Target class [perm] does not exist` , asegúrate de haber registrado el alias en `bootstrap/app.php` y ejecuta:

```
docker-compose exec app composer dump-autoload
docker-compose exec app php artisan config:clear
```

- Permisos de archivos

- Si ves `Failed to open stream: Permission denied` al compilar vistas, ejecuta:

```
docker-compose exec app chown -R www-data:www-data storage bootstrap/cache
docker-compose exec app php artisan view:clear
```

Información útil para colaboradores

- Evita editar migraciones históricas en repositorio compartido si ya han sido ejecutadas en otros entornos; en su lugar crea migraciones adicionales para cambios de esquema.
- Mantén `ADMIN_EMAILS` actualizado en `.env` para pruebas locales; el seeder y el controlador utilizan `config('app.admin_emails')`.

Sesiones y mantenimiento

- El frontend hace un heartbeat a `/profile/heartbeat` (por defecto cada 60s) cuando `window.__isAuth` es truthy. Esto actualiza `last_seen_at` del usuario.
- El cliente intentará notificar cierre de pestaña con `navigator.sendBeacon` a `POST /sessions/end`.
- En servidor hay un comando para cerrar sesiones inactivas:
`php artisan sessions:close-stale --hours=24`. Está registrado en `app/Console/Commands/CloseStaleSessions.php` y programado en `app/Console/Kernel.php` para ejecución diaria.

Checklist mínimo para un nuevo colaborador (en un entorno limpio)

1. Levantar Docker: `docker-compose up -d --build`
2. Composer dentro del contenedor:
`docker-compose exec app composer install --no-interaction --prefer-dist --optimize-autoloader`
3. Generar iconos y compilar assets: `npm run generate-icons && npm run dev`
4. Migraciones y seeders: `docker-compose exec app php artisan migrate:fresh --seed`

Siguiendo ese orden la app debería arrancar en la mayoría de entornos de desarrollo. Si algo falla: copia la salida de la terminal y lo revisamos.

Checklist detallado (PowerShell / Windows — comandos listos para pegar)

Si trabajas en Windows con PowerShell, copia y pega estos comandos en el orden indicado. Ajusta rutas/env si es necesario.

```
# 1) Clona el repositorio y entra en la carpeta
git clone <repo-url>
cd PsicoTest

# 2) Copia el .env de ejemplo y edítalo según tu entorno
cp .env.example .env
# (abrir .env en tu editor para revisar DB, APP_URL, etc.)

# 3) Levanta contenedores (construye si es necesario)
docker-compose up -d --build

# 4) Instalar dependencias PHP dentro del contenedor y optimizar autoload
docker-compose exec app composer install --no-interaction --prefer-dist --optimize-autoloader
docker-compose exec app composer dump-autoload -o

# 5) Generar iconos y compilar assets (localmente o dentro del contenedor)
npm install
npm run generate-icons
npm run dev

# 6) Generar clave de la aplicación (si no existe)
docker-compose exec app php artisan key:generate

# 7) Ejecutar migraciones y seeders (esto reinicia la BD en desarrollo)
docker-compose exec app php artisan migrate:fresh --seed

# 8) Limpiar cachés (opcional pero recomendado)
docker-compose exec app php artisan config:clear
docker-compose exec app php artisan cache:clear
docker-compose exec app php artisan view:clear

# 9) Ajustar permisos si hay errores de escritura (Linux en contenedor)
docker-compose exec app chown -R www-data:www-data storage bootstrap/cache || true
docker-compose exec app chmod -R ug+rw storage bootstrap/cache || true

# 10) Iniciar servidor de desarrollo (opcional, si no usas el dev server de Docker)
# (por defecto la app estará accesible en la URL configurada en APP_URL)
```

Si sigues estos pasos en un entorno limpio, la aplicación debería arrancar. Si algo falla, pega aquí la salida de la terminal y lo reviso.

Preguntas frecuentes

- ¿Debo ejecutar `composer install` en mi host o dentro del contenedor? R: Lo más consistente es hacerlo dentro del contenedor con `docker-compose exec app composer install`.
- ¿Cómo reproduzco el entorno de producción localmente? R: Ajusta `APP_ENV` y `APP_DEBUG` en `.env`, y asegúrate de ejecutar migrations y seeders.

¿Quieres que añada una sección "Comandos útiles (atajos)" más detallada con alias de PowerShell o scripts de npm? Dime qué prefieres y lo agrego.

- [Redberry](#)
- [Active Logic](#)

Contributing

Thank you for considering contributing to the Laravel framework! The contribution guide can be found in the [Laravel documentation](#).

Code of Conduct

In order to ensure that the Laravel community is welcoming to all, please review and abide by the [Code of Conduct](#).

Security Vulnerabilities

If you discover a security vulnerability within Laravel, please send an e-mail to Taylor Otwell via taylor@laravel.com. All security vulnerabilities will be promptly addressed.

License

The Laravel framework is open-sourced software licensed under the [MIT license](#).

Variables de entorno (.env)

Esta sección describe para qué sirve cada variable y en qué categoría cae. Copia `.env.example` a `.env` y ajusta valores para tu entorno local; no compartas secretos reales.

Aplicación

- `APP_NAME`: Nombre de la app (para vistas, correos, títulos).

- APP_ENV: Entorno de ejecución (local, staging, production).
- APP_KEY: Clave de cifrado de Laravel. Obligatoria en producción.
- APP_DEBUG: Activa logs y trazas detalladas en errores (true/false).
- APP_URL: URL pública base de la aplicación.
- APP_LOCALE / APP_FALLBACK_LOCALE / APP_FAKER_LOCALE: Idioma por defecto, idioma de reserva y locale de Faker.
- APP_TIMEZONE: Zona horaria de la app.
- APP_MAINTENANCE_DRIVER / APP_MAINTENANCE_STORE: Mecanismo de “modo mantenimiento”.
- PHP_CLI_SERVER_WORKERS / BCRYPT_ROUNDS: Ajustes de servidor embebido PHP y coste de hash.

Logging

- LOG_CHANNEL / LOG_STACK / LOG_DEPRECATED_CHANNELS / LOG_LEVEL: Configuración de logging y nivel de detalle.

Base de datos

- DB_CONNECTION / DB_HOST / DB_PORT / DB_DATABASE / DB_USERNAME / DB_PASSWORD: Conexión a la BD (pgsql/mysql/sqlite).

Sesiones

- SESSION_DRIVER: Driver de sesión (database/redis/file).
- SESSION_LIFETIME: Minutos de duración de sesión.
- SESSION_ENCRYPT: Cifrado del payload de sesión (true/false).
- SESSION_PATH / SESSION_DOMAIN: Alcance de la cookie de sesión.

Archivos, colas y caché

- FILESYSTEM_DISK: Disco por defecto (local/s3/etc.).
- QUEUE_CONNECTION: Conexión de colas (sync/redis/etc.).
- CACHE_STORE / CACHE_DRIVER / CACHE_PREFIX: Ajustes de caché.
- REDIS_CLIENT / REDIS_HOST / REDIS_PASSWORD / REDIS_PORT: Conexión a Redis.
- MEMCACHED_HOST: Host de Memcached (si aplicase).

Correo

- MAIL_MAILER: Driver de correo (smtp/log/ses/etc.).
- MAIL_HOST / MAIL_PORT / MAIL_USERNAME / MAIL_PASSWORD / MAIL_ENCRYPTION: Config SMTP.

- MAIL_FROM_ADDRESS / MAIL_FROM_NAME: Remitente por defecto.

Administración (semillas/desarrollo)

- ADMIN_EMAILS: Lista CSV de emails con rol admin durante seeders.
- ADMIN_PASSWORD: Password de admin para entorno de desarrollo.
- HR_EMAILS: Lista CSV de emails con rol RRHH (si aplica).

AWS (opcional)

- AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / AWS_DEFAULT_REGION: Credenciales/Región.
- AWS_BUCKET / AWS_USE_PATH_STYLE_ENDPOINT: Bucket y modo de path.

Vite

- VITE_APP_NAME: Nombre de app expuesto al cliente.

Broadcast y WebSockets

- BROADCAST_DRIVER / BROADCAST_CONNECTION: Driver y conexión de broadcast.
- PUSHER_APP_ID / PUSHER_APP_KEY / PUSHER_APP_SECRET / PUSHER_APP_CLUSTER / PUSHER_HOST / PUSHER_PORT / PUSHER_SCHEME: Compatibilidad con Pusher.
- REVERB_APP_ID / REVERB_APP_KEY / REVERB_APP_SECRET / REVERB_HOST / REVERB_PORT / REVERB_SCHEME / REVERB_SERVER_HOST / REVERB_SERVER_PORT / REVERB_SERVER / REVERB_SCALING_ENABLED: Config de Reverb (WebSockets nativo de Laravel).
- VITE_BROADCAST_DRIVER / VITE_REVERB_APP_KEY / VITE_REVERB_HOST / VITE_REVERB_PORT / VITE_REVERB_SCHEME: Variables expuestas a cliente para conectar a websockets.

ConnectyCube (RTC)

- CONNECTYCUBE_APP_ID / CONNECTYCUBE_AUTH_KEY / CONNECTYCUBE_API_KEY / CONNECTYCUBE_AUTH_SECRET / CONNECTYCUBE_DEFAULT_PASSWORD: Credenciales e identidad de la app.
- CONNECTYCUBE_API_ENDPOINT / CONNECTYCUBE_CHAT_ENDPOINT: Endpoints por región (API/XMPP).

Presencia y sesión de navegador

- PRESENCE_ONLINE_SECONDS: Umbral (en segundos) para considerar “online” por recencia.

- `BROWSER_TOKEN_COOKIE_NAME` / `BROWSER_TOKEN_TTL_DAYS`: Token de navegador y TTL.
- `BROWSER_TOKEN_STRICT_UA` / `BROWSER_TOKEN_STRICT_IP`: Restringir token por User-Agent/IP.
- `SESSION_REOPEN_GRACE_SECONDS`: Ventana de gracia para reabrir sesión.

Twilio (opcional)

- `TWILIO_SID` / `TWILIO_TOKEN` / `TWILIO_FROM`: Envío de SMS/2FA.

Nota: No compartas `.env` reales. Usa `.env.example` como base y rellena tus credenciales localmente.